F06｜怎样让另一位操作者完成验收与交接

交接要让下一位操作者取得同样明确的输入与判断依据：**在哪运行、操作谁、预期留下什么、失败停在哪里、之后怎样恢复或暂停。** runbook 是这条操作接口，UAT 是其中对客户路线的验证。

验收要观察什么，为什么重跑可能改变现场？

当前问题	关系或短用法	记到什么程度
UAT 只看首页或内部函数吗？	本例依次请求 live、ready、创建与读回；核对工单编号、状态、设备、症状、创建时间及请求关联。	按客户可见任务选证据，不用端口可达代替保存正确。
一次 UAT 为什么有四个 request ID？	四次 HTTP 请求分别取 LIVE、READY、CREATE、READ 后缀；前缀归组，不把它们合成一次请求。	认熟“验收场景”与“本次请求”的不同粒度。
`data_classification=synthetic` **能证明没用真数据吗？**	它是显式输入合同，不会自动识别任意文字的真实来源。	本例仍只使用自己的合成输入，也未实现登录系统。
创建返回 T-42，就可以跳过读回吗？	后续 GET 还要与创建返回的全部关键字段一致；相同编号可能对应已变化内容。	**结果编号与内容一致性分别看。**
UAT 失败了，直接再跑一遍可以吗？	可能已经创建后才在字段或读回处失败；本例无创建幂等，重跑 POST 会新增工单。	先看失败 step、实际返回编号和现存记录，再确定下一步。不能把重跑当继续上次。

完整输入和四次交换回看「先写客户要看到的结果」；入口 `uat.py` 的完整参数和副作用回看「从全新环境完成一次连续验收」。

一条操作说明，怎样使失败停在正确位置？

当前问题	关系或短用法	记到什么程度
“运行 Compose，确认成功”缺了什么？	缺实验目录、project、端口、等待条件/上限、成功输出与失败处理。	能把动作写到实际对象和可观察结果上。
原生命令失败，为什么后面仍可能继续？	PowerShell 需按相应流程判断 `$LASTEXITCODE`；本例非 0 就抛错停在该步骤。	逐渐认熟退出码用途；不要只看终端最后一行。
等几秒够不够？	等待应围绕 ready 等可观察条件，设预算；到期不发后续轮询，晚返回的成功也不计预算内通过。	分清有界等待与固定睡眠；单次底层 I/O 仍有自己的 timeout 语义。
数据库 count=0 就是首次基线吗？	还需下一编号 T-42；曾创建后删除可能留下不同序列。baseline 只读核对。	**干净基线由实际数据与序列共同确定。** 不为变绿清空未知 volume。

回看：「一条操作说明怎样做到可执行」。

多条验收路线怎样共享数据，又怎样保持独立？

当前问题	关系或短用法	记到什么程度
首次正常路线留下什么？	已核对空基线，创建/读回 T-42，状态 capacity_confirmed；D1/C1 与请求记录可追踪。	样例编号来自这份基线，不能套到任意现存数据库。
替换 API 后验什么？	新实例 ID、同一 volume、原 T-42 全部关键字段仍在。	验实例替换与旧数据关系，不重新造一张代替。
原环境容量降级后有什么变化？	新建 T-43/manual_review，API 仍 ready；恢复容量后原库有两张票，B1 下一编号 T-44。	用独立请求前缀并保留实际状态变化。
隔离恢复创建 T-44，会消耗原环境编号吗？	新 volume 有自己的序列；隔离创建 T-44/manual_review，原库仍两张，下一编号仍 T-44。	**编号相同不代表同一数据库或同一事实。**
D2 回滚后怎样接着验证？	本例数据没迁移；回到精确 D1/C1 后，原环境再建 T-44 并验证正常路线。	根据已发生的步骤取下一编号；完整实验有依赖顺序，不能随意重复。

前置检查、构建、正常、替换、降级、备份、隔离恢复、D2/回滚与交接的完整顺序，回看「从全新环境完成一次连续验收」。长命令按 RUNBOOK 所在阶段查用；不要从不同阶段各取一条拼接。

最终交给别人什么，暂停时又会留下什么？

当前问题	关系或短用法	记到什么程度
只有源码版本为什么不够？	还需实际 image ID、构建来源、Compose/project/配置指纹、端口与挂载、schema/volume 身份、B1 和恢复结果。	**交接的是可识别的程序、配置、数据组合。** 指纹不能替代原文件。
所有验证都写“通过”行不行？	分别写单元/主机双进程/容器/客户路线/恢复/另一操作者完成到哪里。	进程内测试与静态 Compose 检查不证明真实容器、网络、挂载或文档可操作。
ready 通过但读回错，怎样交接？	保留目标身份、请求 ID 与字段差异；客户路线仍失败。备份 hash 不符则停止升级并保留原正确数据。	失败出口接回具体证据，不以局部绿色状态盖过业务失败。
`stop`、普通 `down`、`down --volumes` 差在哪？	stop 保留实例；普通 down 删除本 project 的容器和网络，保留本例 named volume；带 volumes 会扩大到数据卷。	**数据删除范围必须单独认清。** 先看精确对象及 label，再作已决定的暂停或结束操作。
一个项目出错，可以用 prune 清一遍吗？	prune 面向更广的未使用资源，范围超出当前一次故障；F01 独立容器也不属于此 Compose project。	按明确对象处理，不靠全局清理代替定位。
共享清单要贴全部环境和日志吗？	只保留核对所需字段，另记操作者、时间、失败联系人与证据位置。	日志和 inspect 输出可能含敏感内容；信息够判断即可。

回看：「交接清单为什么同时保存程序、配置和数据」「失败出口与日常停止同样属于交付」。另一位满足前置的操作者实际完成路线，才增加“文档足以交接”的证据；阅读或维护这份辅助表不构成运行成功或学习掌握的记录。

跨章查找：构建输入不清回 F02；请求到不了回 F03 的端口；替换后数据不见回 F03 的挂载；live/ready 不同回 F04；旧程序配哪份数据回 F05；步骤仍需猜测回 F06。multi-stage、registry 等扩展只在项目需要时进入，见原教材「什么时候再进入构建与分发扩展」。版本敏感选项在实际操作时从「官方查用入口与版本边界」核对。